

★ CONFIGURE PARALLELISM IN ANSIBLE USING FORKS:

- When Ansible processes a playbook, it runs each play in order.
- After determining the list of hosts for the play, Ansible runs through each task in order.

Normally, all hosts must successfully complete a task before any host starts the next task in the play.

- Ansible could simultaneously connect to all hosts in the play for each task.
- This works fine for small lists of hosts.
- But, if the play targets hundreds of hosts, this can put a heavy load on the control node.
- The maximum number of simultaneous connections that Ansible makes is controlled by the forks parameter in the Ansible configuration file.
- It is set to 5 by default, which can be verified using one of the following:

```
```bash
grep forks ansible.cfg forks = 5
ansible-config dump |grep -i forks
ansible-config list |grep -i forks
```
```

- For example, assume an Ansible control node is configured with the default value of five forks and the play has ten managed hosts.
- Ansible will execute the first task in the play on the first five managed hosts, followed by a second round of execution of the first task on the other five managed hosts.
- After the first task has been executed on all the managed hosts, Ansible will proceed with executing the next task across all the managed hosts in groups of five hosts at a time. Ansible will do this with each task in turn until the play ends.
- The default for forks is set to be very conservative.
- If your control node is managing Linux hosts, most tasks will run on the managed hosts and the control node has less load.
- In this case, you can usually set forks to a much higher value, possibly closer to 100, and see performance improvements.

- If your playbooks run a lot of code on the control node, then you should raise the fork limit judiciously.
- If you use Ansible to manage network routers and switches, then most of those modules run on the control node and not on the network device.
- For network managed hosts,ansible run modules on control nodes and then special connections methods are used to communicate with network devices, like CLI over SSH, XML over SSH or API over HTTP(s)
- Because of the higher load this places on the control node, its capacity to support increases in the number of forks will be significantly lower than for a control node managing only Linux hosts.
- You can override the default setting for forks from the command line in the Ansible configuration file.
- Both the ansible and the ansible-playbook commands offer the -f or --forks options to specify the number of forks to use

★ ROLLING UPDATES:

- Normally, when Ansible runs a play, it makes sure that all managed hosts have completed each task before starting the next task. After all managed hosts have completed all tasks, then any notified handlers are run.
- However, running all tasks on all hosts can lead to undesirable behavior.
- For example, if a play updates a cluster of load balanced web servers, it might need to take each web server out of service while the update takes place.
- If all the servers are updated in the same play, they could all be out of service at the same time.
- To avoid this problem is to use the serial keyword to run the hosts through the play in batches.
- Each batch of hosts will be run through the entire play before the next batch is started.

```
``yaml
```

```
- name: Rolling update
```

```
  hosts: webservers
```

```
  serial: 2
```

```
  tasks:
```

```
    - name: latest apache httpd package is installed
```

```
      yum:
```

```
        name: httpd
```

```
        state: latest
```

```
      notify: restart apache
```

```
  handlers:
```

```
    - name: restart apache
```

```
      service:
```

```
        name: httpd
```

```
        state:
```

```
          restarted
```

```
````
```

- Suppose the webservers group in the previous example contains five web servers that reside behind a load balancer.
- With the serial parameter set to 2, the play will run up to two web servers at a time. Thus, a majority of the five web servers will always be available.
- *For certain purposes, each batch of hosts counts as if it were a full play running on a subset of hosts. This means that if an entire batch fails, the play fails, which causes the entire playbook run to fail.*
- *In the previous scenario with serial: 2 set, if something is wrong and the play fails for the first two hosts processed, then the playbook will abort and the remaining three hosts will not be run through the play. This is a useful feature because only a subset of the servers would be unavailable, leaving the service degraded rather than down.*

### Example to make it crystal clear

- Inventory = 20 hosts
- forks = 10 serial = 4
- What happens:
- Ansible picks **4 hosts** (because of serial)
- Runs tasks on those 4 hosts using **up to 10 forks**  
→ but only 4 hosts exist in the batch
- When batch 1 finishes, it moves to the next 4 hosts
- So:
- forks = max parallel threads
- serial = how many hosts per batch
- **serial always limits forks**

### With forks = 2 and serial = 4, here is exactly what happens

#### Batch 1 (hosts 1–4)

- Runs on 2 hosts → H1, H2
- When one finishes → H3 starts
- When another finishes → H4 st
- Batch completes when all 4 are done